Medical Computer Vision Lab

CI/CD, Containerization, and MLOps Practices: Lab Standard

Maintainer: Ashery Mbilinyi

Version 1.0 : October 16, 2026

Contents

1 Purpose **3**

2 Key Concept Summary (One Page) **3**
2.1 Core Definitions . . . 3
2.2 The Relationship (Mental Model) . . . 3

3 Containerization in ML **3**
3.1 What Containerization Solves . . . 3
3.2 Docker: Packaging and Reproducible Execution . . . 4
3.2.1 Minimum Docker Standard (Lab Policy) . . . 4
3.3 Kubernetes: Orchestration for Scale . . . 4
3.3.1 When Kubernetes is Relevant for the Lab . . . 4
3.3.2 When Kubernetes is Not Necessary . . . 4

4 CI/CD in ML (What It Means in Practice) **5**
4.1 CI: Continuous Integration . . . 5
4.1.1 Typical CI Checks in ML Projects . . . 5
4.2 CD: Continuous Delivery / Deployment . . . 5
4.2.1 CD in ML Has Two Meanings . . . 5
4.2.2 Safe Deployment Pattern . . . 5

5 MLOps: Full Model Lifecycle Management **6**
5.1 Why MLOps Is Needed . . . 6
5.2 Core Components of MLOps . . . 6
5.2.1 (1) Data Management . . . 6
5.2.2 (2) Experiment Tracking . . . 6
5.2.3 (3) Model Checkpointing and Model Registry . . . 6
5.2.4 (4) Automated Pipelines . . . 7
5.2.5 (5) Deployment and Serving . . . 7
5.2.6 (6) Monitoring . . . 7

6 End-to-End Workflow (How Everything Connects) **7**
6.1 Standard Flow . . . 7
6.2 What This Achieves . . . 8

7 Lab Policy Checklist **8**
7.1 Minimum Requirements (All Projects) 8
7.2 Recommended Requirements (Research at Scale) 8
7.3 Optional (When Running Services) 8

8 Appendix: Minimal Templates **8**
8.1 Example: Minimal GitHub Actions CI 8
8.2 Example: CD Builds and Pushes Docker Image 9
8.3 Example: Minimal Dockerfile for ML 9

1 Purpose

This document defines how we operationalize machine learning (ML) systems in the MCV Lab by combining:

- **CI/CD** (automation of testing, integration, and delivery)
- **Containerization** (Docker, Kubernetes) for portability and scalability
- **MLOps practices** for full lifecycle model management

The goal is to ensure our ML work is:

- **Reproducible**: experiments can be rerun months later with identical setup
- **Reliable**: training and inference are stable and testable
- **Deployable**: models can move from research to production systematically
- **Maintainable**: changes are reviewed, tested, and traceable

2 Key Concept Summary (One Page)

2.1 Core Definitions

- **CI (Continuous Integration)**: automatically test and validate code changes on every push/PR.
- **CD (Continuous Delivery/Deployment)**: automatically package and deliver validated code to environments (staging/prod).
- **Containerization**: package the runtime environment so code runs consistently across machines.
- **Docker**: toolchain to build/run containers locally or on servers.
- **Kubernetes (K8s)**: orchestration platform to run containers reliably across a cluster.
- **MLOps**: practices/tools for managing the end-to-end lifecycle of ML models (data → training → evaluation → deployment → monitoring).

2.2 The Relationship (Mental Model)

- **Docker** answers: *"How do we package and run this ML system reproducibly?"*
- **Kubernetes** answers: *"How do we run this at scale and keep it alive reliably?"*
- **CI/CD** answers: *"How do we automate testing and delivery when code changes?"*
- **MLOps** answers: *"How do we manage models like real software products?"*

3 Containerization in ML

3.1 What Containerization Solves

ML projects often break due to environment mismatches:

- different CUDA versions
- different PyTorch versions
- missing system libraries (OpenCV, libjpeg, etc.)

- dependency conflicts across projects

A container solves this by packaging:

- OS-level dependencies
- Python dependencies
- runtime settings and entrypoint scripts

3.2 Docker: Packaging and Reproducible Execution

Docker is used to:

- build container images from a `Dockerfile`
- run training/evaluation in isolated environments
- support consistent onboarding of new lab members

3.2.1 Minimum Docker Standard (Lab Policy)

Every ML project repository should include:

- a `Dockerfile`
- pinned dependencies (`requirements.txt`, `poetry.lock`, or `environment.yml`)
- documented run commands in `README.md`

3.3 Kubernetes: Orchestration for Scale

Kubernetes (K8s) manages running containers across a cluster. It enables:

- scheduling jobs across multiple nodes
- automatic restart/reschedule if a node fails
- autoscaling (e.g., increase number of inference replicas)
- rolling updates (deploy new version with zero downtime)

3.3.1 When Kubernetes is Relevant for the Lab

Kubernetes is most useful when:

- running a shared GPU cluster platform
- serving models as APIs used by others
- running continuous pipelines with multiple services (training + registry + evaluation + serving)

3.3.2 When Kubernetes is Not Necessary

Kubernetes is often **not required** when:

- training is run on single machines or HPC nodes
- the lab uses SLURM scheduling (academic default)
- no persistent production model serving is needed

4 CI/CD in ML (What It Means in Practice)

4.1 CI: Continuous Integration

CI is an automation pipeline triggered by:

- every commit / push
- every pull request (PR)

CI ensures that the codebase stays healthy and deployable.

4.1.1 Typical CI Checks in ML Projects

- **Code quality**: linting (ruff/flake8), formatting (black), type-checking (mypy)
- **Unit tests**: dataset loaders, transforms, model forward pass
- **Training sanity tests**: run 5–20 iterations on tiny data and confirm:
  - no runtime crash
  - loss decreases
  - outputs have expected shapes
- **Reproducibility tests** (optional but recommended):
  - fixed seed yields consistent results for small runs

4.2 CD: Continuous Delivery / Deployment

CD runs after CI passes. It automates:

- building Docker images
- pushing images to registries (GHCR, DockerHub, GCP Artifact Registry)
- deploying to staging
- optionally deploying to production

4.2.1 CD in ML Has Two Meanings

In ML, delivery can refer to:

- **Code delivery**: deploy training/inference software
- **Model delivery**: promote and deploy a specific trained model artifact

4.2.2 Safe Deployment Pattern

- deploy new version to staging
- run smoke tests + basic inference tests
- deploy gradually (canary rollout) when possible
- keep rollback path (`model_vX` → `model_v(X-1)`)

5 MLOps: Full Model Lifecycle Management

5.1 Why MLOps Is Needed

ML differs from classic software:

- models depend on data quality and distributions
- performance can degrade over time (data drift)
- "bugs" may appear only as metric drops
- retraining is often required

MLOps manages ML as a **continuous lifecycle**, not a one-time training script.

5.2 Core Components of MLOps

5.2.1 (1) Data Management

- version datasets and splits (train/val/test)
- document preprocessing and transforms
- prevent leakage across splits
- record dataset metadata and provenance

5.2.2 (2) Experiment Tracking

Every run must log:

- configuration (hyperparameters, seed)
- metrics (loss, AUROC, F1, etc.)
- code version (git commit hash)
- artifacts (plots, confusion matrix)

Recommended tools:

- TensorBoard (baseline)
- Weights & Biases (W&B)
- MLflow

5.2.3 (3) Model Checkpointing and Model Registry

We distinguish:

- **Checkpoints**: intermediate training states (`last.ckpt`, `epoch_10.ckpt`)
- **Model version**: approved/published model artifact with metadata

A model registry tracks:

- which model is in production
- which dataset/training config produced it
- evaluation metrics and approval status

5.2.4 (4) Automated Pipelines

Automate training + evaluation:

- "train" stage
- "evaluate" stage
- "promote model" stage if metrics pass

Typical tools:

- Airflow, Kubeflow Pipelines, Vertex AI Pipelines

5.2.5 (5) Deployment and Serving

Deploying a model means running inference:

- batch inference jobs (HPC or cloud)
- online inference (REST/gRPC service)
- integration into clinical/research pipelines

Typical deployment tools:

- FastAPI + Docker (lab internal)
- BentoML / TorchServe / Triton (advanced)
- KServe / Seldon (K8s-native)

5.2.6 (6) Monitoring

Production monitoring includes:

- latency (response time)
- error rates
- input data drift
- prediction drift
- degradation in downstream clinical performance (when labels exist)

6 End-to-End Workflow (How Everything Connects)

6.1 Standard Flow

1. Researcher proposes change (new model / preprocessing / metric).
2. Code is pushed as a PR.
3. **CI runs**: unit tests + sanity training + linting.
4. If CI passes: **CD builds** a Docker image and publishes it.
5. Image is deployed to staging (optional in academic setting).
6. Training jobs run using:
    - Docker/Apptainer image (environment)
    - SLURM/K8s (execution scheduling)
7. Outputs logged to tracking system (TensorBoard/W&B/MLflow).
8. Best model is promoted into registry and deployed.
9. Monitoring detects drift and triggers retraining when needed.

6.2 What This Achieves

- fewer broken experiments
- fewer wasted GPU hours
- faster onboarding
- traceable results for publications
- reliable path from research to deployment

7 Lab Policy Checklist

7.1 Minimum Requirements (All Projects)

- Training runs without code edits (CLI + YAML/JSON configs)
- Logs saved per run (metrics + config + command)
- Checkpoints saved (`last` + `best`)
- Deterministic seed option exists
- Container recipe exists (`Dockerfile`)

7.2 Recommended Requirements (Research at Scale)

- CI pipeline (GitHub Actions recommended)
- Automated training sanity tests in CI
- Experiment tracking (W&B or MLflow)
- Model registry practice (artifact versioning)

7.3 Optional (When Running Services)

- staging environment for deployments
- monitoring for drift/performance
- Kubernetes-based serving when multi-tenant

8 Appendix: Minimal Templates

8.1 Example: Minimal GitHub Actions CI

Create: `.github/workflows/ci.yml`

```
name: CI

on:
  push:
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
```

```
        with:
          python-version: "3.10"

      - name: Install dependencies
        run: |
          pip install -r requirements.txt

      - name: Lint
        run: |
          python -m compileall .

      - name: Unit tests
        run: |
          pytest -q
```

8.2 Example: CD Builds and Pushes Docker Image

```
name: CD

on:
  push:
    branches: [ "main" ]

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build Docker image
        run: docker build -t mcv-project:latest .
```

8.3 Example: Minimal Dockerfile for ML

```
FROM pytorch/pytorch:2.3.1-cuda12.1-cudnn8-runtime

WORKDIR /workspace

RUN apt-get update && apt-get install -y --no-install-recommends \
    git curl ca-certificates \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt /workspace/requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

COPY . /workspace

CMD ["python", "scripts/train.py", "--help"]
```